TECHNICAL PROJECT REPORT

SOG Gaussian Splatting Unity Plugin

Bringing PlayCanvas SOG format to Aras's UnityGaussianSplatting

---

Author Olli Huttunen

Repository github.com/ollihuttunen/Unity-SOG_plugin

Target platform Unity 6 (6000.0+), Windows x64, URP

Date April 2026

Status Working — full scene import verified

Contents

CHAPTER 01

1. Project Starting Point and Prerequisites

1.1 The Technology: 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) is a real-time rendering technique introduced in 2023 that represents a scene as a collection of millions of semi-transparent 3D ellipsoids — called *Gaussian splats* — rather than polygons. Each splat encodes position, orientation, scale, color (including view-dependent spherical harmonics), and opacity. The result is photo-realistic rendering of real-world scenes at interactive framerates.

The reference implementation outputs a **PLY file**, which can easily exceed 1–2 GB for a high-quality scene. This raw size makes PLY impractical for web delivery, mobile, and many game engine use cases.

1.2 The Goal

The project goal was to build a Unity UPM (Unity Package Manager) plugin that allows developers to import **.sog files** directly into Unity 6 and render them using Aras Pranckevičius's existing UnityGaussianSplatting plugin — without having to first decompress or convert the SOG file externally.

The intended workflow was:

```
Drag .sog file → Unity Project window → Auto-import → GaussianSplatAsset → Assign to GaussianSplatRenderer → Rendered in scene (real-time)
```

1.3 Prerequisites and Dependencies

The project required understanding and integrating three separate systems:

Component	Role	Source
Aras Pranckevičius's UnityGaussianSplatting	Renders Gaussian splats in Unity; defines GaussianSplatAsset binary format	github.com/aras-p/UnityGaussianSplatting
PlayCanvas SOG format	The compressed source format to be decoded	developer.playcanvas.com (spec), github.com/playcanvas/splat-transform (code)
libwebp (Google)	Native library to decode lossless VP8L WebP images inside the SOG archive	chromium.googlesource.com/webm/libwebp
Newtonsoft.Json	Parse meta.json inside the SOG archive	Built-in via Unity Package Manager

> **Key constraint**
> Unity's built-in `ImageConversion.LoadImage` does not support VP8L lossless WebP — the compression format SOG uses for all its image data. This forced a native P/Invoke wrapper around Google's libwebp library for the editor import path.

1.4 Development Environment

- Unity 6 (version 6000.4.2f1), Universal Render Pipeline (URP)
- Windows 11, DirectX 12
- C# / .NET runtime inside Unity (no external build tools)
- Claude Code (Claude Sonnet 4.6) as the AI coding assistant

CHAPTER 02

2. The Role of Aras's Source Code

2.1 What UnityGaussianSplatting Provides

Aras Pranckevičius's open-source UnityGaussianSplatting plugin is the rendering backbone of this project. It provides two critical components that we do not need to re-implement:

- **GaussianSplatRenderer —** a MonoBehaviour that takes a `GaussianSplatAsset` and renders it efficiently on the GPU using compute shaders and custom HLSL.
- **GaussianSplatAsset —** a ScriptableObject that holds references to binary buffer files (`TextAsset`) containing the per-splat data in a specific packed format.

Our plugin's sole responsibility is to read a SOG file and *produce* the exact binary buffers that `GaussianSplatAsset` expects. Once those buffers exist, Aras's renderer handles everything else — sorting, culling, rasterisation, and SH shading.

2.2 Reverse-Engineering the Binary Format

The binary buffer format is not separately documented; it had to be derived by reading Aras's HLSL shader code (`GaussianSplatting.hlsl`) and the C# asset class (`GaussianSplatAsset.cs`). The format for VeryHigh / Float32 quality consists of four buffers:

Buffer	Content	Size per splat
`posData`	3 × float32 position (x, y, z)	12 bytes
`otherData`	10.10.10.2 packed rotation + 3 × float32 scale	16 bytes
`colorData`	4 × float32 (SH0·C0+0.5, SH0·C0+0.5, SH0·C0+0.5, opacity) — Morton-tiled	16 bytes per texel
`shData`	15 × float3 higher-order SH + 1 × float3 padding	192 bytes

2.3 The 10.10.10.2 Quaternion Format

The rotation encoding required careful analysis of Aras's HLSL `DecodeRotation` function. It uses a "smallest-three" scheme: the quaternion component with the largest absolute value is dropped, and the remaining three are stored at 10-bit precision. The dropped component index is stored in 2 bits. The HLSL decoder applies three possible component permutations (`q.wxyz` , `q.xwyz` , `q.xywz`) depending on which component was omitted.

```
// Aras's HLSL decoder (GaussianSplatting.hlsl)
float4 DecodeRotation(float4 pq) {
    uint idx = (uint)round(pq.w * 3.0);
    float4 q;
    q.xyz = pq.xyz * sqrt(2.0) - (1.0 / sqrt(2.0));
    q.w = sqrt(1.0 - saturate(dot(q.xyz, q.xyz)));
    if (idx == 0) q = q.wxyz;
    if (idx == 1) q = q.xwyz;
    if (idx == 2) q = q.xywz;
    return q;
}
```

2.4 Morton-Tiled Color Texture

One non-obvious detail is that color data is stored in a 2048-pixel-wide virtual texture using Z-order (Morton) tiling with 16×16 tiles. The per-splat color is not at a linear offset; a C# port of the HLSL `SplatIndexToPixelIndex` function was required to compute the correct byte offset for each splat's color entry.

2.5 What We Built on Top

Our plugin adds the *input side* that Aras's plugin does not have: a `ScriptedImporter` for `.sog` files, a parser, a converter, and a post-processor that assembles the final asset. Aras's plugin is untouched; it is simply a dependency.

CHAPTER 03

3. How the PlayCanvas SOG Format Is Constructed

3.1 Overview

SOG (Spatially Ordered Gaussians) is a format developed by PlayCanvas to store Gaussian splat scenes in a highly compressed form suitable for web delivery. A `.sog` file is a standard ZIP archive containing a JSON metadata file and a set of WebP images. The entire format fits in 15–20 MB for scenes that would otherwise be 300–500 MB as PLY.

```
.sog (ZIP archive) ├── meta.json ← splat count, codebooks, file names ├── means_l.webp ← position lower bytes (16-bit split) ├── means_u.webp ← position upper bytes ├── quats.webp ← rotations (smallest-three encoding) ├── scales.webp ← scale indices into 256-entry codebook ├── sh0.webp ← DC color + opacity ├── shN_centroids.webp ← higher-order SH cluster centres └── shN_labels.webp ← per-splat cluster index
```

3.2 Position Encoding

Positions are stored with 16-bit precision per axis, split across two WebP images (lower and upper bytes). Before quantisation, a **log transform** is applied:

```
logVal = sign(x) * log(|x| + 1)
```

This transform compresses the dynamic range of large scenes (where most splats cluster near the origin) and improves quantisation accuracy. The `meta.json` `mins` and `maxs` values for the means are in *log space*, not world space. Decoding requires the inverse transform:

```
x = sign(logVal) * (exp(|logVal|) - 1)
```

> **Critical detail — the most impactful bug in this project**
>
> Applying a linear lerp between meta.json mins and maxs, without inverting the log transform, compresses the entire scene toward the origin. A 100-metre scene appears as a 5-unit blob. This was the root cause of the "squishing toward edges" artifact observed early in development.

3.3 Rotation Encoding

Rotations use a **smallest-three quaternion encoding** stored in a WebP image. Each pixel represents one splat:

- **RGB channels**: the three non-largest quaternion components, mapped from [−√2/2, +√2/2] to [0, 255].
- **Alpha channel**: `252 + dropped_index`, indicating which component (0=qw, 1=qx, 2=qy, 3=qz) was omitted.

PlayCanvas stores quaternion components in **[qw, qx, qy, qz]** order internally. The dropped component index follows this convention, which differs from Unity's own quaternion layout and required careful case analysis during decoding.

3.4 Scale Encoding

Scales use a **palette / codebook** approach: a 256-entry float array in `meta.json` holds possible log-scale values. Each splat's scale is stored as three 8-bit indices (R=x, G=y, B=z) into this codebook. This gives 256 possible scale values per axis — sufficient because scale distributions in real scenes are highly clustered. The stored values are log-scales and must be exponentiated before use.

3.5 Color and Opacity (SH0)

The DC (zeroth-order) spherical harmonic coefficients and opacity are stored in `sh0.webp`:

- **RGB channels**: 8-bit codebook indices for the R, G, B color coefficients.

- **Alpha channel**: *direct* linear opacity in [0, 1] — simply `alpha_byte / 255`. It is *not* a codebook index and *not* a logit value.

3.6 Higher-Order Spherical Harmonics (ShN)

Higher-order SH (bands 1–3, up to 15 coefficient vectors per splat) uses a **two-level palette**:

1. `shN_labels.webp` : each splat pixel stores a 16-bit cluster index (R + G<<8).
2. `shN_centroids.webp` : each cluster occupies `ceil(totalCoeffs/3)` pixels; each pixel holds 3 codebook indices (R, G, B channels).

The codebook (256 floats in `meta.json`) dequantises the centroid values. This scheme is analogous to vector quantisation: similar SH profiles are grouped into clusters, and most splats share a cluster rather than storing unique SH data.

For bands = 3: `totalCoeffs = (3+5+7)×3 = 45` floats per cluster, stored as 15 pixels × 3 channels. Coefficients are ordered as interleaved RGB per SH basis function: `[sh1.r, sh1.g, sh1.b, sh2.r, sh2.g, sh2.b, …, sh15.r, sh15.g, sh15.b]`.

CHAPTER 04

4. How SOG and SOGS Differ

> **These are completely unrelated formats**
>
> Despite similar names, SOG and SOGS are created by different organisations, use different compression approaches, and target different use cases. Confusing them early in the project led to incorrect research and wasted effort.

Property	SOG (PlayCanvas)	SOGS (Fraunhofer)
Full name	Spatially Ordered Gaussians	Self-Organizing Gaussians
Organisation	PlayCanvas (web game engine company)	Fraunhofer Heinrich Hertz Institute (research)
Container	ZIP archive (.sog)	Custom binary format (.sogs)
Image encoding	Lossless WebP (VP8L)	PNG or other lossless formats
Compression method	Log-transform quantisation + codebook palettes + two-level SH clustering	Vector quantisation / k-means clustering of Gaussian attributes
Primary use case	Web and game engine delivery (PlayCanvas engine)	Academic / research compression benchmark
Quaternion order	[qw, qx, qy, qz] — PlayCanvas internal convention	Different convention
Opacity storage	Direct linear [0,1] in alpha channel	Logit / sigmoid format
Tooling	playcanvas/splat-transform (open source)	Fraunhofer research tools

4.1 Why the Confusion Matters

The naming similarity caused the project to initially fetch documentation and source code for SOGS (Fraunhofer) when researching SOG (PlayCanvas). The formats share the same high-level concept — compressing 3DGS data using quantisation — but their binary layouts, coordinate conventions, and opacity encodings are entirely different.

Using SOGS documentation to implement a SOG decoder would produce a non-functional result. The correction — switching to PlayCanvas's own documentation and the `splat-transform` open-source encoder/decoder as the ground truth — was essential to making progress.

4.2 Shared Foundations

Both formats build on the same 3DGS foundations from the original 2023 paper by Kerbl et al.: each Gaussian has a position, rotation quaternion, three scale values, an opacity, and spherical harmonic coefficients. The coordinate system conventions (e.g., which axis is "up") also differ between formats and from Unity's own conventions, requiring careful per-format transformation logic.

CHAPTER 05

5. Applying PlayCanvas SOG to Aras's Unity Framework

5.1 The Full Pipeline

```
.sog file | ▼ SOGImporter (ScriptedImporter) | Opens ZIP, calls SOGParser | ▼ SOGParser | Reads meta.json | Decodes WebP images via libwebp P/Invoke | Produces SOGRawData (per-splat arrays) | ▼ SOGConverter | Transforms coordinate system (Y-flip) | Encodes rotations to Aras's 10.10.10.2 format | Applies SH coefficient sign fixes | Writes posData, otherData, colorData, shData (.bytes files) | ▼ SOGPostprocessor (AssetPostprocessor) Detects generated .bytes files Creates GaussianSplatAsset (.asset) Links TextAsset references | ▼ GaussianSplatRenderer (Aras's plugin) Reads binary buffers on GPU Renders splats in real time
```

5.2 Coordinate System Transformation

One of the most complex parts of the project was correctly transforming the SOG coordinate system (Y-axis pointing down) into Unity's coordinate system (Y-axis pointing up). This affects three independent data streams:

Positions

The Y component is simply negated: `(x, -y, z)`.

Quaternions

A naive approach — negating `qy` — is mathematically incorrect and was the source of the "spiky Gaussians" artifact observed during development. The correct transformation is derived from the rotation matrix relationship $\mathbf{R_{Unity}} = \mathbf{M_Y} \cdot \mathbf{R_{SOG}} \cdot \mathbf{M_Y}$ where M_Y = diag(1, −1, 1). Solving the resulting system of equations for quaternion components yields:

```
// WRONG: new Quaternion(qx, -qy, qz, qw)
// CORRECT:
new Quaternion(-qx, qy, -qz, qw)   // negate qx and qz, not qy
```

The sign of `qy` is preserved; `qx` and `qz` are negated. This was verified by solving the matrix equation component-by-component and by visual confirmation in Unity.

Spherical Harmonic Coefficients

SH coefficients were encoded in the original (Y-down) scene coordinate system. Aras's shader evaluates them using the Unity world-space view direction (Y-up). To bridge this mismatch, SH basis functions with *odd parity in Y* must be negated — these are the basis functions that include an odd power of the y coordinate:

Index	Symbol	Basis function	Action
0	sh1	$Y_{1,-1} \sim y$	Negate
1–2	sh2, sh3	$\sim z$, $\sim x$	Keep
3	sh4	$Y_{2,-2} \sim xy$	Negate
4	sh5	$Y_{2,-1} \sim yz$	Negate
5–7	sh6–sh8	even in y	Keep
8	sh9	$Y_{3,-3} \sim y(3x^2-y^2)$	Negate
9	sh10	$Y_{3,-2} \sim xyz$	Negate
10	sh11	$Y_{3,-1} \sim y(\ldots)$	Negate
11–14	sh12–sh15	even in y	Keep

5.3 Lessons Learned — The Bug Journal

Each major visual artifact observed in Unity corresponded to a specific, identifiable bug in the conversion pipeline:

Artifact	Root cause	Fix
Scene compressed toward origin ("squishing")	Missing inverse log-transform on positions	Apply `sign(v)*(exp(|v|)-1)` after lerp
Unity crash on asset assignment	Negative scale values — log-scales stored without `exp()`	Restore `Mathf.Abs(Mathf.Exp(logScale))`
Spiky Gaussians on ground, pointing straight up	Quaternion Y-flip: negating qy instead of qx and qz	Change to `new Quaternion(-qx, qy, -qz, qw)`
Wrong view-dependent shading (additional spikiness)	SH coefficients not adapted for Y-axis flip	Negate SH bands with odd Y parity
Incorrect opacity (too transparent or opaque)	Alpha channel treated as codebook index or logit	Use direct `alpha_byte / 255`

CHAPTER 06

6. Benefits and Opportunities: SOG in Unity

6.1 File Size — The Core Advantage

The most immediate benefit is dramatic file size reduction. The test scene used throughout development (*EetunAukio*, a photogrammetric outdoor scan) demonstrates this clearly:

Format	Approximate size	Splat count
PLY (uncompressed)	~350–500 MB	1.2–1.3 million
SOG (compressed)	~17–20 MB	same
Ratio	**~15–20× smaller**	

For game development and real-time applications, this difference is critical. A 500 MB asset is impractical to ship; an 18 MB asset is easily included in a game build or downloaded over a mobile connection.

6.2 Photorealistic Scene Capture in Unity

3DGS enables capturing real-world locations with photorealistic quality using only a smartphone or camera. With this plugin, that workflow extends into Unity:

1. Capture a location with photos (e.g., a building, historical site, product).
2. Process with a 3DGS tool (Inria, gsplat, Postshot, etc.) → PLY file.
3. Convert to SOG using PlayCanvas's `splat-transform` tool.
4. Drag the `.sog` file into Unity → instant import → render in scene.

This pipeline makes photo-realistic environment capture accessible to Unity developers without specialist 3D modelling skills.

6.3 Use Cases

Architectural Visualisation

Existing buildings, construction sites, or heritage structures can be scanned and brought into Unity as Gaussian splat assets for walkthroughs, presentations, and design reviews — with no manual modelling required.

Digital Twins

Real-world environments can be replicated as lightweight Gaussian splat scenes and embedded in Unity applications for simulation, monitoring, or training data generation.

Cultural Heritage and Museum Applications

High-fidelity scans of artefacts, sculptures, or historic sites can be archived and presented interactively in VR/AR applications built on Unity, with file sizes small enough for real-time streaming.

Game Development — Environmental Assets

Background environments, props, or atmospheric elements captured from reality can supplement hand-crafted 3D art pipelines. A photorealistic courtyard, forest, or streetscape can be captured once and reused across projects.

XR and Spatial Computing

The compact file size of SOG is particularly valuable for XR headsets where storage and bandwidth are constrained. Gaussian splat scenes can be streamed or bundled with applications targeting Apple Vision Pro, Meta Quest, and similar platforms (with platform-specific renderer

adaptations).

6.4 Limitations to Consider

- **Static scenes only —** Gaussian splats represent a frozen capture of a scene. Dynamic objects, animated characters, and real-time lighting interaction are not supported by the current rendering model.
- **View-dependent rendering —** Gaussian splats look best from camera angles close to the original capture direction. Wide departures from the capture viewpoint can produce floating artefacts.
- **GPU memory —** Even compressed, large scenes with millions of splats consume significant GPU buffer memory. Streaming and level-of-detail systems would be needed for very large scenes.
- **Editor-only import (current state) —** The plugin currently requires importing in the Unity Editor before runtime. A fully dynamic runtime loader (loading `.sog` directly at runtime without prior editor import) would require additional engineering.

6.5 Future Directions

Several natural extensions of this work would increase its utility:

- **Cross-platform native WebP —** Porting the libwebp integration to macOS and Linux to enable full cross-platform editor support.
- **Runtime loading —** Streaming SOG files at runtime without prior editor import, enabling procedurally loaded or downloaded scene content.
- **Compressed quality tiers —** Currently only Float32 (highest quality) is used. Supporting Aras's Norm16/Norm11 formats would reduce GPU memory use.
- **Scene composition —** Tooling to place and scale multiple Gaussian splat assets within a Unity scene with proper lighting integration.
- **Other compressed formats —** The same approach could be extended to support SPZ (Niantic/Scaniverse) or other emerging 3DGS delivery formats.

6.6 Conclusion

This project demonstrates that PlayCanvas's SOG format can be faithfully decoded and rendered inside Unity using Aras's GaussianSplatting plugin as the rendering layer. The key engineering challenges were understanding the log-space position encoding, the correct coordinate system transformation for quaternions and spherical harmonics, and navigating Unity's asset pipeline constraints.

The resulting plugin makes it straightforward to bring photorealistic real-world Gaussian splat scenes into Unity 6 at compact file sizes — a capability with broad applicability in games, XR, digital twins, and cultural heritage.

> **Final result**
> A 17.9 MB SOG file containing 1.3 million Gaussian splats imports correctly and renders in Unity 6 at full quality — positions, rotations, scales, colors, and all three bands of spherical harmonics — visually matching the PlayCanvas reference viewer.